GQA and Attention Optimization in IREE

Nishanth Senthil Kumar (EE23B049) Dhruv Prasad (EE23B130)

1. Introduction, Prior Work and Original Proposal

Introduction and problem. Grouped-query attention (GQA) has more query heads than key/value (KV) heads. For example, with $H_Q = 8$ and $H_{KV} = 2$, each KV head serves four query heads. A straightforward frontend repeats the small K and V tensors to match the number of query heads:

$$K_{\text{expanded}} = \text{repeat}(K, 4), \qquad \text{KVhead}(h) = \lfloor h/4 \rfloor .$$

The repetition changes tensor storage, not the attention mathematics. In our early lowered IR, a broadcast dispatch wrote the expanded K/V tensors before subsequent matrix multiplications consumed them.

Original idea and implementation. Recognize `broadcast/expand + reshape + attention` and use shared KV heads directly, rather than writing repeated tensors. A C++/MLIR prototype reorganized the query-head grouping and eliminated the explicit intermediate on representative patterns.

```
// Before: Q [8,S,D], K,V [2,S,D]
K8 = repeat_interleave(K, 4, head_axis);
V8 = repeat_interleave(V, 4, head_axis);
out = attention(Q, K8, V8);
// After: head h reads K[h/4], V[h/4]; no stored K8/V8.
```

Initial observation. An illustrative For $H_Q = 8$, $H_{KV} = 2$, $S = 128$ and $D = 32$ in FP32, explicitly materializing both expanded K and V tensors adds **192 KiB** of tensor storage compared with compact K/V. This is a theoretical storage difference, not a measured runtime allocation or DRAM-traffic reduction.

Prior work: checking what IREE already optimizes

An IREE flag ablation showed that two *existing* passes already remove both separate KV-broadcast dispatches on the tested graphs:

IREE configuration	Separate K/V broadcast dispatches
Neither flag	2
Generalize matmul only	2
Aggressive reshape movement only	2
Both flags	**0**

```
--iree-opt-generalize-matmul
--iree-dispatch-creation-enable-aggressive-reshape-movement
```

While developing our original GQA pass, we tested IREE compilation flags and discovered that enabling matmul generalization together with aggressive reshape movement already removed the redundant K/V expansion in our tested graphs. This made our original rewrite unnecessary for those configurations. Rather than duplicate an existing compiler optimization, we redirected the project toward other opportunities in KV-cache updates, projection placement and attention lowering. This finding also established a stronger IREE baseline for the rest of our experiments. The GQA formulation follows Ainslie et al. [1]; the baseline compiler behavior is documented by IREE [2,3].

2. Proposed Methods and Implementation: KV Cache

During autoregressive inference, a KV cache stores historical tokens' keys/values. Functional tensor graphs may express small modifications as new versions of the entire cache. The compiler can sometimes avoid moving or constructing those full versions.

2.1 Conditional cache-read forwarding

Intuition: If we change one entry in a notebook and immediately look up a different entry, we can read the original notebook instead of copying every page.

Before: update a large cache C at location j, producing $C' = \text{update}(C, j, x)$, then immediately gather $C'[i]$. Naive lowering may materialize C'.

After: forward the read to the new value only if it overlaps the update; otherwise read the old cache:

$$C'[i] = \begin{cases} x, & i = j, \\ C[i], & i \neq j. \end{cases}$$

This matters especially for speculative/forked graphs, where a large cache version can be created for a candidate that is not ultimately used. The transformation specifically targets graphs that express temporary functional cache versions. The rewrite is related to tensor slice semantics and compiler canonicalization [4,5], and to the cache-sharing problems studied by PagedAttention [6].

Result versus IREE: Tesla T4 **1.23× geometric mean across 9 shapes**, best **3.44×**; the benefit was shape-dependent. Representative CPU cases: 1.42–3.63×. **GPU cache-forwarding measurements**

Separate synthetic cache workloads on the Tesla T4 measured latency and total DRAM reads plus writes using Nsight Compute:

Workload	Base ms	Opt. ms	Speedup	DRAM MB
Small sparse forks	0.288	0.246	1.17×	5.831 → 0.128
Large sparse forks	1.777	1.230	1.44×	178.897 → 0.137
Dense no-rewrite control	0.545	1.268	0.43×	0.285 → 0.275

The large sparse case reduced measured DRAM traffic by **99.92%**, with a **1.44×** latency speedup. These are isolated cache workloads. The dense no-rewrite control had equal canonical IR, but its timing direction flipped across repetitions; the cause remains unresolved.

2.2 Masked-gather forwarding

Instead of per-element conditional control flow, this variant gathers from the original cache and the small update, then selects the required values. Here U replaces rows $p : p + t$ of C:

```
inside = (p <= ids) & (ids < p+t)
safe = where(inside, ids-p, 0) # U has at least one row
base = gather(C, ids)
patch = gather(U, safe)
R = where(expand_mask(inside), patch, base)
```

The mask selects U for indices inside the updated interval and C elsewhere. The safe index matters because both sides of a selection may be evaluated: even a discarded patch load must use a valid address.

Two gathers and a mask add work, so a cost check rejects dense demand. An irregular-shape CUDA case failed to compile in the masked form and used the conditional fallback. The conditional and masked forms implement the same law; their speedups must not be added together.

Result. On the CPU A1 cache workload, masked-gather forwarding reduced latency from 187.10 to 74.36 μs, a **2.52×** speedup. Conditional forwarding achieved 2.04× on the same workload. These are alternative implementations, so their gains are not additive. The reported T4 forwarding-suite results do not establish a separate masked-only GPU speedup.

2.3 Update-chain forwarding and coalescing

Intuition: Combine several nearby edits into one operation, just as updating a paragraph at once is simpler than saving a new document after every word.

Before: write several adjacent patches successively, then read from the final version:

```
C1 = update(C, i, x0)
C2 = update(C1, i+1, x1)
y = gather(C2, index)
```

After: coalesce eligible contiguous writes and forward reads directly to the matching patch, falling back to C only when no patch overlaps. This avoids repeatedly treating short intermediate versions as full tensors. The compiler must preserve update order, including overlapping writes. Related compiler foundations include MLIR slice folding and canonicalization [4,5].

Result: Tesla T4 **1.01–1.03**× for 4–16 updates, versus **2.17–2.34**× in CPU tests. The GPU measurements show a modest benefit, while the CPU experiments show larger savings for repeated updates.

2.4 Winner-cache factoring

Intuition: When only one of two proposed changes will be accepted, choose the winning change before copying the shared history.

Before: several speculative branches each construct an updated full cache; only one branch wins:

```
A = update(C, patch_A)
B = update(C, patch_B)
result = select(winner, A, B)
```

After: choose the small patch first and commit once:

```
patch = select(winner, patch_A, patch_B)
result = update(C, patch)
```

Instead of maintaining multiple competing full cache versions, the graph creates only the selected final version. Safe only if branches share the same starting cache and the selection has compatible semantics. Speculative decoding motivates this branch structure [7,8]; our compiler transformation is distinct from their decoding algorithms.

Result: 1.57× in a T4 branch-selection benchmark (CPU: **3.60**×). This concept led to the later full-model *concat-commit factoring* experiment on page 5.

3. Proposed Methods: Projection and Attention

3.1 Latent projection placement

Intuition: Instead of converting thousands of stored compact keys into large vectors, convert the current query into a compact vector once.

Before: for every cached latent vector, materialize a large projected key. If $L \in \mathbb{R}^{T\times r}$, $W_K \in \mathbb{R}^{d\times r}$, and $Q \in \mathbb{R}^{1\times d}$, the score is $Q(LW_K^\top)^\top$.

After: project the single query into the latent dimension and then compare against the compact cache:

$$\underbrace{Q(LW_K^\top)^\top}_{\text{project } T \text{ cache rows}} \quad = \quad \underbrace{(QW_K)L^\top}_{\text{project one query}} \quad .$$

For a long context, this can reduce redundant key projections from T vectors to one query (up to orientation conventions and other kernels). It uses standard matrix associativity; rounding and backend layout can affect correctness/performance. Latent attention and projection absorption are described in DeepSeek-V2 [9].

Result: T4 **1.20**× **geometric mean over 16 shapes**; a broader valid suite reported **1.238**× **geometric mean over 26 cases**, with regressions on some shapes. CPU: **3.93**× geometric mean over 16 shapes.

3.2 Partial-RoPE projection

Background: Rotary Position Embedding (RoPE) encodes token positions in attention by rotating pairs of coordinates in query and key vectors. For a two-dimensional pair, a rotation through angle θ is

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}.$$

The angle depends on token position (with different frequencies for different coordinate pairs), allowing query–key scores to capture relative position. *Partial-RoPE* separates the representation into a non-rotary (plain/content) part and a smaller rotary (position-dependent) part.

Intuition: Only the rotary part requires position-dependent rotation; projection work on the plain part can be reused or moved to the query side.

Before: For each cached token, reconstruct a large projected key, including components that do not require rotation. A position-dependent rotation generally prevents moving an arbitrary key projection through the rotary operation.

After: Handle the two components separately. For a cached compact vector c_t and plain key $k_{\text{plain},t} = W_{\text{plain}}c_t$, use associativity:

$$q_{\text{plain}}^\top(W_{\text{plain}}c_t) = (W_{\text{plain}}^\top q_{\text{plain}})^\top c_t.$$

Thus project the current query once rather than projecting every cached plain key. Keep the position-dependent calculation for the smaller rotary part:

$$\text{score}_t = q_{\text{plain}}^\top k_{\text{plain},t} + q_{\text{rope}}^\top R_t k_{\text{rope},t}.$$

Here R_t denotes the appropriate relative-position rotation (including the effect of the query position). The rewrite does *not* commute arbitrary projections with RoPE rotations. See RoFormer [10] for RoPE and DeepSeek-V2 [9] for decoupled rotary components.

Result: T4 **1.50× geometric mean in the evaluated projection suite**; CPU **11.40×** in the evaluated projection suite. These are isolated test cases, not all-model speedups.

3.3 Native-attention fallback

Intuition: If a specialized attention kernel does not support a particular wide shape reliably, use an equivalent sequence of standard operations for that shape.

Problem: certain wide query/key dimensions (tested widths 256/512) required an alternative lowering path to preserve numerical correctness.

Fix: guard the problematic shape and lower the computation into validated elementary operations (matmuls, softmax, value aggregation), rather than the original native form for those shapes. The fallback preserved the tested result and sometimes compiled to much faster kernels. This is motivated by the native-versus-decomposed lowering trade-off; see IREE pipeline documentation [2] and FlashAttention [11,12] for the broader kernel-design context.

Result: a targeted T4 comparison fell from **14.50 ms to 1.56 ms (9.32×)** with output agreement in subsequent verification. This demonstrates the effectiveness of shape-aware lowering for wide query/key dimensions.

4. Evaluation Against IREE and Memory Analysis

Baseline definition: Every speedup below compares an optimized graph with its corresponding *unoptimized IREE graph* for the same study. Ratios from different studies should not be multiplied or treated as one common end-to-end workload.

Study	Tesla T4 speedup vs IREE	CPU speedup vs IREE
Original GQA repeat elimination	Not isolated on GPU; existing flags remove repeats	1.86× historical test vs default lowering
Cache-forwarding suite	1.23× geom. mean (9); best 3.44×; not separated by implementation	1.42–3.63× representative cases
Conditional read forwarding	No separately attributed result	2.04× on A1 (187.10 → 91.74 μs)
Dense no-rewrite control	0.43× in recorded run; timing direction unstable	Not reported
Update coalescing plus forwarding	1.01–1.03× (4–16 updates)	2.17–2.34× combined
Winner factoring plus forwarding	1.57× branch-selection benchmark	3.60× combined; factoring alone 1.38×
Latent projection placement	1.20× geom. mean (16)	3.93× geom. mean (16)
Partial-RoPE plus projection	1.50× geom. mean (4)	11.40× geom. mean (4)
Native-attention fallback	9.32× targeted comparison	10.63× vs correct guarded lowering

Conditional and masked forwarding are alternative implementations. The suite-level GPU measurements do not establish separate speedups for each implementation. Combined results must not be interpreted as gains from each constituent pass alone. The dense control had equal canonical IR; its timing direction flipped across repetitions, so its slowdown remains unresolved. The targeted GPU fallback comparison is a separate study from the CPU comparison against correct guarded lowering.

What the numbers show

- **Strongest reusable idea:** avoid large, temporary KV-cache versions when only a patch or one selected branch is needed. Forwarding and factoring can reduce unnecessary memory movement.
- **Shape-aware benefits:** gains vary by tensor dimensions, motivating compiler guards that select the better lowering for each workload.
- **CPU gains do not predict GPU gains.** GPU kernel fusion, parallelism, launch overhead and memory behavior change the outcome.
- **Numerical verification:** rewritten outputs were checked against their corresponding IREE computations, and the attention fallback was evaluated separately for wide dimensions.

How we built and tested the transformations

A standalone **C++ MLIR pass tool** performs the pattern rewrites; Python scripts feed IR into IREE, compile both control and rewritten graphs, check outputs, and time CPU/T4 executions. We inspected dispatches and intermediate buffers to distinguish *actual graph changes* from speedups caused only by backend choice. **Measured IREE version:** 3.13.0 / 3.13.0rc20261016 (study-specific builds).

Memory and DRAM profiling

We measured GPU DRAM reads and writes using NVIDIA Nsight Compute on the Tesla T4, alongside compiler dispatch and allocation measurements. In a synthetic forked-cache benchmark, forwarding reduced total DRAM traffic by approximately **99.92%**, demonstrating how avoiding large temporary cache copies reduces off-chip transfers. In the pretrained branching workload, the change in total measured DRAM traffic was **−3.91%** at 2,048 tokens, **+1.24%** at 3,072, and **−0.27%** at 4,096. The latency benefit therefore reflects more than DRAM bytes alone: eliminating duplicate work and changing kernel scheduling also matter. Separately, the original GQA ablation reduced first-invocation allocator bytes from 140,288 to 9,216. These quantities distinguish logical storage, allocations, and hardware-measured memory traffic [11].

Relationship to existing compiler optimizations

Existing IREE passes already address the original GQA repeat pattern, while established projection identities motivate the remaining methods. Our work brings these ideas into an evaluated MLIR rewrite workflow and studies when cache-aware transformations improve IREE-generated code.

5. Full-Model Evaluation, Conclusions and References

Question. Do these synthetic cache ideas help an actual pretrained transformer running under IREE? We evaluated **SmolLM2-135M-Instruct** (30 transformer layers) on a **Tesla T4**, timing one next-token full-model inference step with a custom two-candidate cache-commit graph.

Change. Instead of building/concatenating two candidate cache histories and selecting the result, *select the candidate increment first, then commit once*:

```
// Before // After
A = concat(C, a) patch = select(win, a, b)
B = concat(C, b) out = concat(C, patch)
out = select(win, A, B)
```

This is the model-level **concat-commit factoring** specialization of winner-cache factoring, related to cache sharing in [6–8]. **60 rewrite markers** were present in the rewritten model, versus zero in the control. Other pass families were *not* demonstrated as active in this particular full-model test.

Matched, validated IREE-only comparison

Median latency per next-token step	2,048-token context	4,096-token context
Stable IREE baseline	43.264 ms	82.409 ms
Stable IREE + commit factoring	**31.743 ms**	**77.361 ms**
Speedup	**1.36×**	**1.06×**
Latency reduction	**26.46%**	**5.52%**

Protocol: multiple prompts and repeated synchronized timing after warmup; medians aggregated. Both stable-IREE variants passed numerical validation (absolute tolerance 5×10^{-4}, relative tolerance 3×10^{-3}). We use the numerically validated stable IREE configuration for a matched comparison.

Interpretation. Avoiding competing cache commits saved about **11.45 ms** at 2,048 tokens, but only **4.55 ms** at 4,096. The result shows that removing duplicate cache commits helps a branch-aware inference graph. The metric is next-token step latency on this workload; full-sequence generation throughput is a separate measurement.

6. Conclusion

We began by removing repeated KV-head expansion and established how to achieve it with existing IREE compiler optimizations. Six follow-up investigations located more useful opportunities in **KV-cache dataflow**, **branch commits**, and **projection placement**. The most concrete model-level result is a validated **1.36× IREE speedup at context 2,048** for the custom cache-commit workload, and **1.06× at 4,096**. These experiments show that compiler-level cache restructuring can improve realistic inference graphs while retaining mathematical equivalence.

References and Project Resources

Project code: https://github.com/lazarsalt1441/cs6886_project

[1] J. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," EMNLP, 2023. https://arxiv.org/abs/2305.13245

[2] IREE Project, "Optimization Options." https://iree.dev/reference/optimization-options/

[3] IREE Project, "Aggressive Reshape Movement," pull request 22707. https://github.com/iree-org/iree/pull/22707

[4] LLVM Project, "MLIR Tensor Dialect." https://mlir.llvm.org/docs/Dialects/TensorOps/

[5] LLVM Project, "MLIR Operation Canonicalization." https://mlir.llvm.org/docs/Canonicalization/

[6] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP, 2023. https://arxiv.org/abs/2309.06180

[7] Y. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML, 2023. https://arxiv.org/abs/2211.17192

[8] X. Miao et al., "SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification," 2023. https://arxiv.org/abs/2305.09781

[9] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024. https://arxiv.org/abs/2405.04434

[10] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2021. https://arxiv.org/abs/2104.09864

[11] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS, 2022. https://arxiv.org/abs/2205.14135

[12] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023. https://arxiv.org/abs/2307.08691

[13] Project measurements, "Attention Consolidated Results," October 2026. https://github.com/lazarsalt1441/cs6886_project

[14] Project experiments and reproducibility materials, November 2026. https://github.com/lazarsalt1441/cs6886_project/tree/main/dram